

AI活用による単体テスト作成効率化 PoC提案書

1. 背景

近年、生成AI技術の急速な発展により、ソフトウェア開発工程への活用が進んでいる。

特に、設計書解析、テストケース作成、コードレビューなどの工程においては、AIによる生産性向上および品質向上が期待されている。

一方で、AIによる誤生成(Hallucination)、機密情報の取り扱い、成果物品質の担保など、実運用に向けた課題も存在する。

そのため本提案では、単体テスト作成工程を対象としたPoC(Proof of Concept)を実施し、AI活用による効果および運用上の課題を検証する。

2. 目的

本PoCでは、以下の観点について検証を行う。

2.1 業務効率向上

- 単体テスト作成工数の削減
- 設計書解析作業の効率化
- テスト作成プロセスの標準化

2.2 品質向上

- 設計漏れの早期発見
- テスト観点の網羅性向上
- 設計レビュー品質の向上

2.3 AI活用ノウハウの蓄積

- AI利用ガイドライン整備
- AI活用パターンの確立
- 将来的な適用範囲拡大に向けた知見獲得

3. 対象範囲

3.1 対象工程

工程	実施内容
詳細設計書解析	要件整理・テスト観点抽出
単体テスト項目作成	テストケース作成支援
設計書レビュー支援	記載漏れ・不整合確認

3.2 対象技術

分類	内容
設計書	Excel詳細設計書
開発言語	Java、Go
テストフレームワーク	JUnit、Go testing
AIツール	GitHub Copilot

4. PoC全体構成

Phase1 (PoC対象)

Excel詳細設計書

↓

Agent1 (設計書解析・UT項目生成)

↓

UT項目書 (Markdown)

↓

Agent2 (設計書整合性チェック)

↓

指摘事項一覧

Phase2 (将来構想)

Agent3 (テストケース生成)

↓

Agent4 (UTコード生成)

↓

Agent5 (AIレビュー)

↓

開発者レビュー

↓

Commit

5. Phase1 (PoC対象)

5.1 Agent1: 設計書解析・UT項目生成 Agent

概要

Excel形式の詳細設計書を解析し、単体テスト作成に必要な情報を整理する。

AIへ直接Excelファイルを入力するのではなく、必要情報を抽出してMarkdown形式へ変換し、その内容をGitHub Copilotへ入力することで、テスト項目作成を支援する。

想定ワークフロー

Excel詳細設計書

↓

解析ツール (C# / Go)

↓

Markdown形式へ変換

↓

GitHub Copilot

↓

UT項目書生成

Markdown変換対象

項目

機能ID

機能名称

入力項目

出力項目

業務ルール

エラー条件

制約条件

想定利用技術

用途	技術
Excel解析	OpenXML SDK
Excel解析	ClosedXML
文書生成	Markdown Template
AI支援	GitHub Copilot Chat

期待効果

- テスト項目作成時間の短縮
- 要件整理手順の標準化
- 後続工程への入力品質向上

5.2 Agent2: 設計書整合性チェック Agent

概要

Agent1で生成されたUT項目書を基準として設計書を再確認し、設計漏れや不整合を検出する。

チェック対象

- 正常系処理
- 異常系処理
- 境界値条件
- 入力制約
- エラーコード
- 業務ルール

期待効果

- 設計品質向上
 - レビュー工数削減
 - 設計漏れの早期発見
-

6. セキュリティおよび運用方針

AI利用にあたり、機密情報保護の観点から以下のルールを適用する。

6.1 入力データ制限

- 顧客情報を投入しない
- 個人情報を投入しない
- 本番データを利用しない
- 必要最小限の情報のみ利用する

6.2 運用ルール

- AI投入前に内容を確認する
 - AI生成結果をそのまま採用しない
 - 成果物は必ず人間がレビューする
 - 最終判断は開発者が実施する
-

7. 将来構想 (Phase2以降)

7.1 Agent3: テストケース生成 Agent

役割

- テストケース一覧作成
- 境界値分析
- 異常系整理

成果物

- テストケース一覧
-

7.2 Agent4: UTコード生成 Agent

役割

- JUnitコード生成
- Go単体テストコード生成

運用ルール

- 設計書記載内容のみ利用
- 不明点はTODOとして出力
- 開発者レビュー必須

7.3 Agent5: AIレビュー Agent

役割

- テスト漏れ確認
- 命名規約確認
- 重複コード検出
- 品質チェック

8. 想定効果

項目	期待効果
テスト項目作成	工数削減
設計レビュー	品質向上
設計漏れ検出	早期発見
テスト品質	標準化
AI活用ノウハウ	社内蓄積

※具体的な削減率についてはPoC結果を基に評価する。

9. リスクと対策

リスク	内容	対策
AI誤解釈	設計書に記載されていない内容を推測して生成する可能性がある	設計書記載内容のみ利用し、人的レビューを必須とする
設計書フォーマット差異	プロジェクトごとにExcel形式が異なる	標準テンプレートを整備し解析対象を統一する
情報漏洩	機密情報がAIへ送信される可能性がある	個人情報・顧客情報を除外し利用ルールを整備する
AI依存	開発者の設計理解が低下する可能性がある	相互レビューおよびレビュー必須運用を実施する
ブラックボックス化	AI生成根拠が不明確になる可能性がある	テスト観点および生成根拠を明示する

10. PoC成功判定指標(KPI)

PoCでは以下の指標を測定する。

KPI	測定方法
UT項目作成時間	作業時間計測
レビュー工数	レビュー時間計測
設計漏れ検出件数	指摘件数集計
GitHub Copilot利用回数	利用ログ確認
利用者満足度	アンケート実施

11. 今後の進め方

まずはPhase1として、Agent1およびAgent2を対象にPoCを実施する。

PoC完了後、KPIに基づく効果測定を行い、十分な有効性が確認できた場合は以下の順で段階的に適用範囲を拡大する。

1. テストケース生成支援
2. UTコード生成支援
3. AIレビュー支援

小規模な範囲から導入し、運用ルールおよび品質管理方法を確立した上で、将来的な開発工程全体へのAI活用を目指す。

前回の会議では、既存開発プロセスへのAI活用について簡単にご紹介させていただきました。

今回はその続きとして、AI導入の第一歩として実施可能なPoC(概念実証)案をご説明いたします。

AIを開発工程全体へ一度に導入することは、品質管理や運用ルール整備の面で課題があります。そのため、まずはリスクが低く効果測定しやすい単体テスト(UT)作成プロセスを対象として、小規模に導入効果を検証することを提案します。

今回の提案では、設計書解析からテスト項目作成、設計書レビュー支援までを対象とし、実際の利用結果や課題を確認しながら、今後の本格導入可否を判断することを目的としています。

まずは小さく始め、効果が確認できた段階で対象範囲を拡大していく方針です。